

Guía de Exposición — Frontend (Angular 19)

Documento de Sustentación Técnica para Exposición

UNAMConnect — Portal de Tutorías Académicas

Esta guía está diseñada para respaldar la explicación técnica del **Frontend** durante la exposición ante el jurado o docente. Contiene la fundamentación arquitectónica, justificación de tecnologías, desglose detallado de todos los componentes, servicios, guards e interceptores, y un guion estructurado paso a paso para la demostración.

1. Introducción y Arquitectura del Frontend 🚀

El frontend de **UNAMConnect** es una Aplicación de Página Única (**SPA - Single Page Application**) desarrollada sobre **Angular 19**, orientada a brindar una experiencia de usuario fluida, reactiva e intuitiva.

Principios Arquitectónicos Aplicados:

- Componentes Standalone:** Se prescindió del uso de `NgModule` tradicionales para adoptar la arquitectura *Standalone* nativa de Angular 19. Esto reduce la complejidad del proyecto, mejora la modularidad y optimiza los tiempos de carga inicial mediante *Lazy Loading* (carga diferida).
- Reactividad basada en Signals:** La gestión de estado se realiza con **Angular Signals** (`signal()` , `computed()`), lo cual elimina la necesidad de ciclos de detección de cambios pesados (Zone.js) y garantiza actualizaciones ultrarrápidas de la interfaz de usuario.
- Inyección de Dependencias Modular:** Los servicios (`AuthService` , `FirestoreService` , `NotificationService`) se inyectan en la raíz del árbol (`providedIn: 'root'`), garantizando una única instancia de estado (*Singleton*).
- Diseño Visual Responsivo y Moderno:** Implementado con **Bootstrap 5** y estilos personalizados en **Vanilla SCSS/CSS**, combinando estética limpia (*Glassmorphism*, gradientes de alto contraste) y accesibilidad.

2. Tecnologías Aplicadas y Justificación Técnica 🛠️

Durante la exposición, el docente preguntará: **¿Por qué eligieron estas tecnologías para el Frontend?** A continuación se detalla la respuesta técnica justificada:

1. Angular 19 & TypeScript

¿Por qué se eligió?: Angular ofrece un marco de trabajo completo (*Full-Featured Framework*) con tipado estático estricto a través de **TypeScript**. Esto evita errores comunes en tiempo de ejecución, facilita la refactorización segura y proporciona contratos de datos claros (`interfaces`) coincidiendo con la API REST del backend.

Beneficio Clave: Soporte nativo para *Signals*, enrutamiento con *Guards*, interceptores HTTP y componentes encapsulados.

2. Signals de Angular (Gestión de Estado)

¿Por qué se eligió?: A diferencia del enfoque tradicional de RxJS puro o librerías pesadas como NGRX para estados simples, los *Signals* permiten declarar variables reactivas granulares. Cuando un *Signal* cambia (ej: abrir modal de chat), solo el nodo afectado del DOM se vuelve a renderizar, mejorando dramáticamente el rendimiento.

3. Bootstrap 5 & Vanilla SCSS/CSS

¿Por qué se eligió?: Bootstrap proporciona una grilla responsiva probada (`row` , `col-md-X`) y utilidades visuales ágiles. Se combinó con **Vanilla SCSS** para construir un sistema de diseño propio con paletas tailoring (gradientes azules `#00c6ff` a `#0072ff` , efectos de cristal, avatares de alto contraste y botones pills).

4. RxJS & Interceptores HTTP

¿Por qué se eligió?: Permite manejar operaciones asíncronas complejas de manera declarativa. Con los interceptores HTTP (`HttpClient`), inyectamos automáticamente los tokens de autenticación JWT en cada petición saliente y capturamos errores globales (401, 403, 500) en un solo lugar.

5. Google Firebase Web SDK (Auth & Storage)

¿Por qué se eligió?: Permite autenticación cliente-servidor delegada y la subida directa de archivos (PDFs de boletas de notas e imágenes) hacia la nube de **Firebase Storage**, reduciendo la carga en nuestro servidor Node.js.

3. Estructura del Proyecto Frontend

```
frontend/src/app/
├── components/           # Componentes globales reutilizables
│   ├── confirm-dialog/  # Modal emergente de confirmación de acciones
│   └── loader/          # Spinner de carga global asíncrono
├── config/              # Archivos de configuración
│   ├── bypass-emails.config.ts # Lista de correos de prueba autorizados
│   ├── curriculum.config.ts   # Malla curricular y lista de carreras/cursos
│   └── firebase.config.ts     # Inicialización de Firebase Web SDK
├── guards/              # Protectores de rutas
│   ├── auth.guard.ts      # Verifica sesión activa
│   ├── guest.guard.ts     # Evita acceso a login si ya tiene sesión
│   └── role.guard.ts       # Valida rol asignado (alumno/tutor/moderador)
└── interceptors/        # Interceptores de peticiones HTTP
```

```

|   ├── auth.interceptor.ts      # Inyecta token JWT en Authorization Header
|   └── error.interceptor.ts     # Captura de errores HTTP globales
└── models/                     # Interfaces TypeScript
    ├── usuario.model.ts        # Modelo de datos de usuario
    ├── curso.model.ts          # Modelo de asignatura
    ├── asesoria.model.ts       # Modelo de asesoría programada
    └── rol.model.ts            # Modelo de roles
└── pages/                      # Vistas y Páginas de la Aplicación
    ├── auth/                  # Módulo de Autenticación
    │   ├── login/             # Pantalla de Inicio de Sesión + Carousel
    │   ├── register/          # Pantalla de Registro de Alumno
    │   └── forgot-password/    # Recuperación de Contraseña
    └── dashboard/             # Dashboards por Rol
        ├── student/           # Dashboard Principal del Alumno (+ subcomponentes)
        ├── tutor/              # Dashboard del Tutor (Horarios 24h, Peticiones)
        └── moderator/         # Dashboard del Moderador (Verificación de Boletas)
└── services/                  # Servicios Inyectables (Lógica de Negocio)
    ├── auth.service.ts        # Autenticación y control de tokens
    ├── firebase.service.ts     # Subida de PDFs a Firebase Storage
    ├── notification.service.ts # Alertas emergentes (Toasts) y Modales
    ├── loader.service.ts       # Estado del indicador de carga
    └── theme.service.ts        # Control de temas visuales

```

4. Desglose Detallado de Componentes (Component por Component)

4.1. Componentes Base y Estructura Global

AppComponent (`app.ts`):

Función: *Componente raíz* (Shell* de la aplicación).

* **Operación:** Escucha los estados del `LoaderService` y `NotificationService` para renderizar el spinner de carga centralizado y las alertas emergentes (Toasts) por encima de cualquier vista.

4.2. Módulo de Autenticación (`pages/auth`)

LoginComponent (`login.ts` / `login.html`):

* **Función:** Maneja el ingreso de usuarios registrados.

* **Operación:** Valida que el correo termine en `@unam.edu.pe`, realiza la petición de login al backend Express, guarda el token JWT en `localStorage` y redirige al dashboard del rol del usuario (`/dashboard`, `/dashboard/tutor`, o `/dashboard/moderator`).

CarouselComponent (`carousel.component.ts`):

* **Función:** Componente visual informativo presente en las pantallas de Login y Registro. Muestra testimonios y diapositivas de la plataforma.

RegisterComponent (`register.ts` / `register.html`):

* **Función:** Formulario reactivo multietapa para la creación de nuevas cuentas de alumnos.

* **Operación:** Enforma reglas estrictas: Código Universitario obligatorio de 10 dígitos numéricos, correo institucional obligatorio `@unam.edu.pe`, selección de carrera profesional y contraseña segura.

ForgotPasswordComponent (`forgot-password.component.ts`):

* **Función:** Permite solicitar el restablecimiento de clave mediante un enlace enviado al correo del estudiante.

4.3. Dashboard del Alumno (`pages/dashboard/student`)

El dashboard de Alumno es el corazón de la experiencia estudiantil. Está compuesto por un controlador principal (`student.ts`) y subcomponentes modulares:

StudentDashboardComponent (`student.ts` / `student.html`):

* **Función:** Administra el estado principal del alumno y el menú de navegación lateral.

Operación: Maneja Signals* para controlar la pestaña activa (`activeTab`), carga los datos del alumno autenticado y orquesta el **Modal Rápido de Chat** (`mostrarModalChat`) para iniciar conversaciones directas con cualquier tutor.

StudentCoursesComponent (`student-courses.ts` / `student-courses.html`):

* **Función:** Muestra la malla curricular universitaria organizada por ciclos (Ciclo I al Ciclo X).

* **Operación:** Permite al alumno explorar los cursos de su carrera y ver qué tutores están disponibles para una asignatura específica.

StudentTutorsComponent (`student-tutors.ts` / `student-tutors.html`):

* **Función:** Catálogo dinámico de tutores autorizados.

* **Operación:** Filtra la lista de tutores por nombre, materia o carrera profesional. Renderiza las tarjetas de tutores mostrando avatares estilizados (foto de Firebase o iniciales blancas sobre gradiente azul de alto contraste), calificación por estrellas y botones de acción rápida: "+ Asesoría" y "Mensaje".

StudentAdvisoriesComponent (`student-advisories.ts` / `student-advisories.html`):

* **Función:** Gestión y seguimiento de asesorías solicitadas.

* **Operación:** Muestra las citas agendadas agrupadas por estado:

* **Pendiente** : Esperando que el tutor confirme.

* **Aceptada** : Muestra el botón directo "Unirse a Google Meet" con el enlace virtual único.

* **Rechazada** : Muestra la razón brindada por el tutor.

* **Modal de Valoración:** Permite asignar de 1 a 5 estrellas y escribir una reseña tras culminar la clase.

StudentChatComponent (`student-chat.ts` / `student-chat.html`):

* **Función:** Centro de mensajería en tiempo real.

* **Operación:** Muestra la lista de conversaciones activas con los tutores, el historial de mensajes enviados/recibidos y actualiza el contador de mensajes no leídos.

StudentProfileComponent (`student-profile.ts` / `student-profile.html`):

* **Función:** Perfil del estudiante y postulación a tutor.

* **Operación:** Muestra la información personal del alumno y contiene el formulario de **Postulación a Tutor**, permitiendo seleccionar materias dominadas y subir la Boleta de Notas en PDF.

StudentStatsComponent & **StudentStatsTabComponent** :

* **Función:** Visualización gráfica de métricas personales del alumno (horas de asesoría recibidas, materias reforzadas).

4.4. Dashboard del Tutor (`pages/dashboard/tutor`)

TutorDashboardComponent (`tutor.ts` / `tutor.html`):

* **Función:** Centro de control para el tutor académico.

* **Operaciones Clave:**

1. **Disponibilidad Horaria 24h:** Registra bloques de atención seleccionando el día (Lunes a Sábado). Ofrece botones de **Bloques Frecuentes (1 Clic)** (08:00 - 10:00 , 10:00 - 12:00 , 14:00 - 16:00 , 16:00 - 18:00) y selecciones manuales en formato de 24 horas (07:00 a 22:00). Actualiza en tiempo real la tabla matricial de 6 días.

2. **Atención de Peticiones:** Revisa las solicitudes entrantes. Al presionar "**Aceptar**", invoca la API para generar el enlace dinámico de Google Meet.

3. **Toggles de Cursos:** Permite activar o desactivar las materias que desea mostrar activamente en el catálogo.

4. **Gestión de Recursos:** Sube archivos PDF y guías de estudio para sus alumnos.

4.5. Dashboard del Moderador (`pages/dashboard/moderator`)

ModeratorDashboardComponent (`moderator.ts` / `moderator.html`):

* **Función:** Panel de administración y auditoría del sistema.

* **Operaciones Clave:**

1. **Visor de Boletas de Notas:** Revisa las postulaciones de tutores pendientes y permite visualizar/descargar el documento PDF o imagen adjunta en un modal.

2. **Aprobación de Roles:** Aprueba al estudiante promoviéndolo a Tutor y activando sus cursos, o rechaza la solicitud indicando la causa.

3. **Administración de Usuarios y Malla Curricular:** Edición de cuentas, reseteo de claves y adición de carreras/cursos.

4. **Panel KPIs:** Visualiza el total de tutorías dictadas, alumnos atendidos y satisfacción general.

5. Servicios, Guards e Interceptores 🛡️

5.1. Servicios (`services/`)

AuthService : Almacena el usuario autenticado en una *Signal*, gestiona el token JWT en `localStorage` , realiza peticiones de login/registro y destruye la sesión (`logout`).

FirestoreService : Conecta con Firebase Storage Web SDK para la subida asíncrona de archivos (PDFs de boletas, imágenes de perfil) retornando la URL pública segura.

NotificationService : Expone métodos reactivos (`showSuccess()` , `showError()` , `confirm()`) para disparar avisos flotantes o modales de confirmación.

LoaderService : Activa o desactiva el spinner global durante las transiciones HTTP.

5.2. Guards de Navegación (`guards/`)

authGuard : Impide que usuarios anónimos accedan a rutas protegidas (`/dashboard`).

roleGuard : Lee la propiedad `expectedRole` de la ruta y verifica que el rol del usuario autenticado coincida (ej: si un alumno intenta ingresar a `/dashboard/moderator` , lo redirige a su propio panel).

guestGuard : Impide que un usuario que ya inició sesión vuelva a visualizar las páginas de Login o Registro.

5.3. Interceptores HTTP (`interceptors/`)

auth.interceptor.ts : Clona cada petición saliente hacia la API del backend e inyecta la cabecera:

`Authorization: Bearer`

error.interceptor.ts : Escucha respuestas de error HTTP del backend. Si recibe un `401 Unauthorized` (token vencido), limpia el storage y redirige automáticamente al usuario al Login.

6. Guion de Exposición para el Estudiante (Demostración del Frontend) 🗣️

A continuación tienes la guía paso a paso para explicar el Frontend durante la demostración en vivo:

Paso 1: Bienvenida e Inicio de Sesión (1 min)

"Buenas tardes profesor/jurado. Comenzaremos demostrando el Frontend de UNAMConnect desarrollado en Angular 19. Como pueden observar en la pestaña del navegador, hemos personalizado la experiencia con el título institucional

UNAMConnect - Portal de Tutorías Académicas . Ingresaremos con una cuenta de estudiante registrada con correo institucional @unam.edu.pe ."

Paso 2: Exploración del Catálogo y Filtros (2 mins)

"Al ingresar al Dashboard del Alumno, la interfaz reactiva basada en Angular Signals carga la información de inmediato sin recargar la página. En el 'Catálogo de Tutores' podemos filtrar por asignatura o carrera. Observen el diseño visual: los avatares de los tutores combinan sus fotos de perfil o sus iniciales en blanco sobre un gradiente azul de alto contraste para garantizar accesibilidad."

Paso 3: Solicitud de Asesoría y Modal Rápido de Chat (3 mins)

"Si el alumno desea consultar algo antes de agendar, puede hacer clic en el botón 'Mensaje' en la tarjeta del tutor. Se despliega un modal emergente que permite enviar un mensaje directo. Para agendar una asesoría, presionamos '+ Asesoría', seleccionamos la fecha y el horario disponible configurado por el tutor en formato de 24 horas. Al confirmar, la solicitud pasa al estado Pendiente."

Paso 4: Demostración del Panel de Tutor y Google Meet (3 mins)

"Cambiemos a la vista del Tutor. En la sección 'Registrar Nuevo Bloque', el tutor puede configurar su disponibilidad horaria de Lunes a Sábado utilizando nuestros atajos de 1-clic para bloques frecuentes (como 08:00 a 10:00) o dropdowns personalizados de 24 horas. En 'Solicitudes Entrantes', al presionar 'Aceptar', el sistema genera automáticamente un enlace de videoconferencia de Google Meet que se comparte de inmediato con el alumno."

Paso 5: Módulo de Moderación y Auditoría de Boletas (2 mins)

"Por último, en el panel del Moderador, la administración puede evaluar las postulaciones de los estudiantes que desean ser tutores. Al hacer clic en 'Ver Boleta de Notas', nuestro visor integrado abre el archivo PDF subido a Firebase Storage para verificar el rendimiento académico antes de otorgar el rol de Tutor."

Paso 6: Conclusión Técnica (1 min)

"Gracias al uso de Angular 19 Standalone Components, Signals e Interceptores HTTP con Firebase Storage, hemos logrado una aplicación web sumamente rápida, modular, segura y libre de recargas molestas de página."